

Building Websites Using ASP.NET Core Razor Pages

Objectives

- ◆ Overview ASP.NET Core Razor Pages
 - ◆ Introduce Razor Page files and Razor syntax
 - ◆ Work with Page Models (Handler Methods, ViewData, Action Results)
 - ◆ Understand Tag Helpers in Razor Pages
 - ◆ Understand and work with View Components
 - ◆ Apply Routing and URLs in Razor Pages
 - ◆ Apply and configure the Startup with Razor Pages
 - ◆ Validate input data with Validation and Model Binding in Razor Pages
 - ◆ Understand and work with State Management in Razor Pages
 - ◆ Scaffolding for Razor Pages
 - ◆ Demo

ASP.NET Core Razor Pages - 1

- ◆ ASP.NET Razor Pages is a server-side, page-focused framework that enables building dynamic, data-driven web sites with clean separation of concerns.
- ◆ Part of the ASP.NET Core web development framework from Microsoft, Razor Pages supports cross platform development and can be deployed to Windows, Unix and Mac operating systems.
- ◆ The Razor Pages framework is lightweight and very flexible.
 - ◆ It provides the developer with full control over rendered HTML.
 - ◆ Razor Pages is the recommended framework for cross-platform server-side HTML generation.

ASP.NET Core Razor Pages - 2

- ◆ Razor Pages makes use of the popular C# programming language for server-side programming, and the easy-to-learn Razor templating syntax for embedding C# in HTML mark-up to generate content for browsers dynamically.
- ◆ Architecturally, Razor Pages is an implementation of the MVC pattern and encourages separation of concerns.
- ◆ Razor Pages is included within .NET Core from version 2.0 onwards, which is available as a free download as either an SDK or a Runtime.

Razor Pages

- ◆ Razor is a markup syntax for embedding server-based code into webpages.
- ◆ The Razor syntax consists of Razor markup, C#, and HTML. Files containing Razor generally have a .cshtml file extension.
- ◆ The default Razor language is HTML. Rendering HTML from Razor markup is no different than rendering HTML from an HTML file. HTML markup in .cshtml
- ◆ Razor files is rendered by the server unchanged.
- ◆ Different types of ASP.NET Razor
 - ◆ Razor Page (Single Page Model) – Demo1.cshtml
 - ◆ Razor Page (with Page Model) – Demo2.cshtml + Demo2.cshtml.cs
 - ◆ Razor with MVC

Razor Pages – Single File Approach

- ◆ The Razor code block is denoted by an opening `@{` and is terminated with a closing `}`. The content within the block is standard C# code.
- ◆ Functions blocks

```
@page
@functions {
    [BindProperty]
    public string Name { get; set; }
}
<div style="margin-top:30px;">
    <form method="post">
        <div>Name: <input name="name" /></div>
        <div><input type="submit" /></div>
    </form>
    @if (!string.IsNullOrEmpty(Name))
    {
        <p>Hello @Name!</p>
    }
</div>
```

```
@page
@{
    var name = string.Empty;
    if (Request.HasFormContentType)
    {
        name = Request.Form["name"];
    }
}
<div style="margin-top:30px;">
    <form method="post">
        <div>Name: <input name="name" /></div>
        <div><input type="submit" /></div>
    </form>
</div>
<div>
    @if (!string.IsNullOrEmpty(name))
    {
        <p>Hello @name!</p>
    }
</div>
```

Razor Pages – PageModel Files

- Any code relating to the processing of user input or data should be placed in PageModel files, which share a one-to-one mapping with their associated content page.
- They even share the same file name, albeit with an additional .cs.

```
@page
@model RazorPagesDemo.Pages.Example1Model
@{
}

<div style="margin-top:30px;">
    <form method="post">
        <div>Name: <input asp-for="Name" /></div>
        <div><input type="submit" /></div>
    </form>
    @if (!string.IsNullOrEmpty(Model.Name))
    {
        <p>Hello @Model.Name!</p>
    }
</div>
```

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.RazorPages;

namespace RazorPagesDemo.Pages
{
    5 references
    public class Example1Model : PageModel
    {
        [BindProperty]
        0 references
        public string Name { get; set; }
        0 references
        public void OnGet()
        {
        }
    }
}
```

Different types of Razor files - 1

- ◆ Razor files have a leading underscore (_) in their file name. These files are not intended to be browsable.
- ◆ The ***__Layout.cshtml*** file acts as a template for all content pages that reference it. Consistent parts of a site's design are declared in this file.
- ◆ The ***__ViewStart.cshtml*** file contains code that executes after the code in any content page in the same folder or any child folders. It provides a convenient location to specify the layout file for all content pages that are affected by it.
- ◆ The ***__ViewImports.cshtml*** file provides a mechanism to make directives available to Razor pages globally so that you don't have to add them to pages individually.

Different types of Razor files - 2

- ◆ Partial Pages or Views are Razor files containing snippets of HTML and server-side code to be included in any number of pages or layouts.
- ◆ Partial pages can be used to break up complex pages into smaller units, thereby reducing the complexity and allowing teams to work on different units concurrently.
- ◆ Partial pages are cshtml files that do not take part in routing. Rendering Partial Pages:
 - ◆ `<partial name="_MenuPartial" />`
 - ◆ `@Html.Partial("_MenuPartial")`
 - ◆ `@{ Html.RenderPartial("_MenuPartial"); }`

Different types of Razor files - 3

- ◆ The layout page acts as a template for all pages that reference it.

_MasterLayout.cshtml

```
<!DOCTYPE html>

<html>
<head>
    <meta name="viewport" content="width=device-width" />
    <title>@ViewBag.Title</title>
</head>
<body>
    <div>
        @RenderBody()
    </div>
</body>
</html>
```

_SubLayout1.cshtml

```
@{
    Layout = "/_MasterLayout";
}
<div class="main-content-one-col">
    @RenderBody()
</div>
```

Different types of Razor files - 4

- ◆ `_ViewImports.cshtml` file provides a mechanism to centralize directives that apply to Razor pages.
- ◆ The default Razor Pages template includes a `_ViewImports.cshtml` file in the Pages folder - the root folder for Razor pages.
- ◆ The `_ViewImports.cshtml` file supports the following directives:
 - ◆ `@addTagHelper`, `@removeTagHelper`, `@tagHelperPrefix`
 - ◆ `@inherits`, `@namespace`
 - ◆ `@inject`, `@model`, `@using`

```
@using RazorPagesDemo
@namespace RazorPagesDemo.Pages
@addTagHelper *, Microsoft.AspNetCore.Mvc.TagHelpers
```

Razor Syntax

- ◆ A Razor content page acts as a template for generating HTML. The typical content page includes static HTML, tag helpers that emit HTML dynamically, and C# code.
- ◆ The C# code is embedded within the static content and tag helpers according to a set of rules, or syntax.
- ◆ All code blocks must appear within `@{ ... C# code }` brackets.
- ◆ Comments within a code block can be denoted by two forward slashes `//`. Alternatively, you can use `/*...*/` or `@*...*`.

The Razor Pages PageModel - 1

- ◆ The main purpose of the Razor Pages PageModel class is to provide clear separation between the UI layer (the .cshtml view file) and processing logic for the page. There are a number of reasons why this separation is beneficial:
 - ◆ It reduces the complexity of the UI layer making it easier to maintain.
 - ◆ It facilitates automated unit testing.
 - ◆ It enables greater flexibility for teams in that one member can work on the view while another can work on the processing logic.
 - ◆ It encourages smaller, reusable units of code for specific purposes, which aids maintenance and scalability.

The Razor Pages PageModel - 2

- ◆ The PageModel class is a combination of a *Controller* and a *ViewModel*.
- ◆ Controllers feature in a number of design and architectural patterns concerned with the presentation layer of an application. They are most commonly found in the Model-View-Controller (MVC) pattern, where the controller can be implemented as a Front Controller or a Page Controller.
- ◆ A front controller is defined as "a controller that handles all requests for a website". A page controller is "an object that handles a request for a specific page or action on a website". A Razor PageModel class is an implementation of the Page Controller pattern.
- ◆ The Page Controller pattern is characterized by the fact that there is a one-to-one mapping between pages and their controllers.

The Razor Pages PageModel - 3

- ◆ A View Model is an implementation of the Presentation Model design pattern.
- ◆ It is a self-contained class that represents the data and behaviour of a specific "view" or page. The view model pattern is used extensively in MVC application development, where it mainly represents data, but typically little behaviour.
- ◆ In Razor Pages, the PageModel is also the view model.
- ◆ Razor Pages is sometimes described as implementing the MVVM (Model, View ViewModel) pattern. It doesn't. The MVVM pattern is applied to applications where the presentation and model share the same layer.

The Razor Pages PageModel - 4

- The following code shows the content that is generated for each file when you use the Razor Page (with page model) option to add a new page to a Razor Pages application

Index1.cshtml

```
@page
@model RazorPagesDemo.Pages.Index1Model
@{
}
```

Index1.cshtml.cs

```
using Microsoft.AspNetCore.Mvc.RazorPages;

namespace RazorPagesDemo.Pages
{
    5 references
    public class Index1Model : PageModel
    {
        0 references
        public void OnGet()
        {
        }
    }
}
```

Handler Methods in Razor Pages - 1

- ◆ Handler methods in Razor Pages are methods that are automatically executed as a result of a request. The Razor Pages framework uses a naming convention to select the appropriate handler method to execute.
- ◆ The name of the method, which is prefixed with "On": OnGet(), OnPost(), OnPut() etc.
- ◆ Handler methods also have optional asynchronous equivalents: OnPostAsync(), OnGetAsync() etc (methods that contain asynchronous code).
- ◆ As far as the Razor Pages framework is concerned, OnGet and OnGetAsync are the same handler.

```
public class IndexModel : PageModel
{
    public string Message { get; set; }
    public void OnGet()
    {
        Message = "Get used";
    }
    public void OnPost()
    {
        Message = "Post used";
    }
}
```

Working With ViewData in Razor Pages - 1

- ◆ ViewData is a container for data to be passed from the PageModel to the content page.
- ◆ ViewData is a dictionary of objects with a string-based key.

```
public class IndexModel : PageModel
{
    public void OnGet()
    {
        ViewData["MyNumber"] = 42;
        ViewData["MyString"] = "Hello World";
        ViewData["MyComplexObject"] = new Book {
            Title = "Sum Of All Fears",
            Author = new Author { Name = "Tom Clancy" },
            Price = 5.99m
        };
    }
}
```

```
@page
@model IndexModel
@{
}
<h2>@ViewData["MyString"]</h2>
<p>The answer to everything is @ViewData["MyNumber"]</p>
```

Working With ViewData in Razor Pages - 2

◆ ViewData Attribute

```
public class IndexModel : PageModel
{
    [ViewData]
    public string Message { get; set; }
    public void OnGet()
    {
        Message = "Hello World";
    }
}
```

```
@page
@model IndexModel
@{
}
<h2>@Model.Message</h2>
<h2>@ViewData["Message"]</h>
```

◆ ViewBag is a wrapper around the ViewData dictionary and provides an alternative way to access ViewData contents within ASP.NET Core MVC controllers using dynamic properties instead of string-based indexes.

```
@page
@model IndexModel
@{
    ViewBag.Title = "My Home Page";
}
```

```
<title>@ViewBag.Title</title>
```

```
<title>@ViewData["Title"]</title>
```

Action Results in Razor Pages - 1

- ◆ Action results in Razor Pages are commonly used as the return type of handler methods and are responsible for generating responses and appropriate status codes.
- ◆ Action results implement either the abstract **Microsoft.AspNetCore.Mvc.ActionResult** class, or the **Microsoft.AspNetCore.Mvc.IActionResult** interface.

Action Results in Razor Pages - 2

- ASP.NET Core includes more than three dozen ActionResult classes covering a wide range of needs, including but not limited to executing and returning the content of a Razor page, returning the content of a file, redirecting to another resource or simply returning a specific HTTP status code.

```
public IActionResult OnGet()  
{  
    return new RedirectToPageResult("Index");  
}
```

```
public RedirectToPageResult OnGet()  
{  
    return new RedirectToPageResult("Index");  
}
```

Tag Helpers - 1

- ◆ Tag helpers are reusable components for automating the generation of HTML in Razor Pages.
- ◆ Tag helpers target specific HTML tags. The ASP.NET Core framework includes a number of predefined tag helpers targeting many commonly used HTML elements as well as some custom tags:
 - ◆ Anchor tag helper, Cache tag helper, Environment tag helper,
 - ◆ Form Action tag helper, Form tag helper, Image tag helper,
 - ◆ Input tag helper, Label tag helper, Link tag helper, Option tag helper
 - ◆ Partial tag helper, Script tag helper, Select tag helper
 - ◆ Textarea tag helper, Validation tag helper, Validation Summary tag helper

Tag Helpers - 2

- ◆ The Tag helpers used in Razor Pages were introduced as part of ASP.NET MVC Core and are found in the **Microsoft.AspNetCore.Mvc.TagHelpers** package which is included as part of the **Microsoft.AspNetCore.All** meta-package.
- ◆ Enabling Tag Helpers
 - ◆ `@addTagHelper *, Microsoft.AspNetCore.Mvc.TagHelpers`
 - ◆ `@addTagHelper "*, Microsoft.AspNetCore.Mvc.TagHelpers"`
- ◆ Selective tag processing - Use the `@addTagHelper` and `@removeTagHelper` directives to opt in or opt out of
 - ◆ `@addTagHelper "*, Microsoft.AspNetCore.Mvc.TagHelpers"`
 - ◆ `@removeTagHelper "Microsoft.AspNetCore.Mvc.TagHelpers.AnchorTagHelper, Microsoft.AspNetCore.Mvc.TagHelpers"`

Tag Helpers - 3

- ◆ The Input tag helper generates appropriate name and id attribute values based on the PageModel property that is assigned to it.
- ◆ It will also generate an appropriate value for the type attribute, based on the property's meta data. The tag helper will also emit attributes that provide support for unobtrusive client-side validation.
- ◆ Type attribute based on data annotations

DataType Enumeration	Attribute	Input type
EmailAddress	EmailAddress	email
PhoneNumber	Phone	tel
Password		password
Url	Url	url
Date		date
Time		time
DateTime, Duration		datetime-local
HiddenInput ¹		hidden

Tag Helpers - 4

```
public class Member
{
    public int PersonId { get; set; }
    public string Name { get; set; }
    [EmailAddress]
    public string Email { get; set; }
    [DataType(DataType.Password)]
    public string Password { get; set; }
    [DataType(DataType.PhoneNumber)]
    public string Telephone { get; set; }
    [Display(Name="Date of Birth")]
    public DateTime DateOfBirth { get; set; }
    public decimal Salary { get; set; }
    [Url]
    public string Website { get; set; }
    [Display(Name="Send spam to me")]
    public bool SendSpam { get; set; }
    public int? NumberOfCats { get; set; }
    public IFormFile Selfie { get; set; }
}
```

```
public class RegisterModel : PageModel
{
    [BindProperty]
    public Member Member { get; set; }
    public void OnGet()
    {
    }
}
```

```
<form method="post">
    <input asp-for="Member.PersonId" /><br />
    <input asp-for="Member.Name" /><br />
    <input asp-for="Member.Email" /><br />
    <input asp-for="Member.Password" /><br />
    <input asp-for="Member.Telephone" /><br />
    <input asp-for="Member.Website" /><br />
    <input asp-for="Member.DateOfBirth" /><br />
    <input asp-for="Member.Salary" /><br />
    <input asp-for="Member.SendSpam" /><br />
    <input asp-for="Member.NumberOfCats" /><br />
    <input asp-for="Member.Selfies" /><br />
    <button>Submit</button>
</form>
```

Code generation

```
<form method="post">
    <input type="number" data-val="true" data-val-required="The PersonId field is required." id="Member_PersonId" name="Member.PersonId" value="" /><br />
    <input type="text" id="Member_Name" name="Member.Name" value="" /><br />
    <input type="email" data-val="true" data-val-email="The Email field is not a valid e-mail address." id="Member_Email" name="Member.Email" value="" /><br />
    <input type="password" id="Member_Password" name="Member.Password" /><br />
    <input type="tel" id="Member_Telephone" name="Member.Telephone" value="" /><br />
    <input type="url" data-val="true" data-val-url="The Website field is not a valid fully-qualified http, https, or ftp URL." id="Member_Website" name="Member.Website" value="" /><br />
    <input type="datetime-local" data-val="true" data-val-required="The Date of Birth field is required." id="Member_DateOfBirth" name="Member.DateOfBirth" value="" /><br />
    <input type="text" data-val="true" data-val-number="The field Salary must be a number." data-val-required="The Salary field is required." id="Member_Salary" name="Member.Salary" value="" /><br />
    <input data-val="true" data-val-required="The Send spam to me field is required." id="Member_SendSpam" name="Member.SendSpam" type="checkbox" value="true" /><br />
    <input type="number" id="Member_NumberOfCats" name="Member.NumberOfCats" value="" /><br />
    <input type="file" id="Member_Selfie" name="Member.Selfie" /><br />
    <button>Submit</button>
    <input name="__RequestVerificationToken" type="hidden" value="cf0381..." />
    <input name="Member.SendSpam" type="hidden" value="false" />
</form>
```

View Components in Razor Pages - 1

- ◆ View Components perform a similar role to Tag Helpers and Partial Pages.
- ◆ View components are recommended instead of partial pages or tag helpers whenever any form of logic is required to obtain data for inclusion in the resulting HTML snippet, specifically calls to an external resource such as a file, database or web service.
- ◆ View components also lend themselves to unit testing.
- ◆ View components are particularly useful for data-driven features in a layout page where there is no related page model or controller class.

View Components in Razor Pages - 2

- ◆ View components consist of a class file and a .cshtml view file. The class file contains the logic for generating the model. It can be thought of as a mini-controller, just as the Razor PageModel file is considered to be a controller. The view file contains the template used to generate the HTML to be plugged in to the page that hosts the view component.
- ◆ The class file must conform to the following rules:
 1. It must derive from the *ViewComponent* class
 2. It must have "ViewComponent" as a suffix to the class name or it must be decorated with the [ViewComponent] attribute
 3. It must implement a method named *Invoke* with a return type of *IViewComponentResult*

Razor Pages Routing

- ◆ Routing is the system that matches URLs to Razor pages (matching URLs to file paths, starting from the root Razor Pages folder)
- ◆ When a Razor Pages application starts up, a collection of Attribute Routes is constructed, using the file and folder paths rooted in the Pages folder as the basis for each route's template.
- ◆ Index.cshtml - you can access Index.cshtml by browsing to both *http://yourdomain.com/* and *http://yourdomain.com/Index*.
- ◆ If you create a folder named Demo and add a file named Index.cshtml to it, a further two routes will be defined with the following templates: "Demo", "Demo/Index"

The Program.cs Class - 1

- ASP.NET Core makes extensive use of dependency injection (DI) - a technique that facilitates loose coupling of code.

```
var builder = WebApplication.CreateBuilder(args);
```

```
// Add services to the container.  
builder.Services.AddRazorPages();
```

```
var app = builder.Build();
```

```
// Configure the HTTP request pipeline.
```

```
if (!app.Environment.IsDevelopment())  
{  
    app.UseExceptionHandler("/Error");  
}
```

```
app.UseStaticFiles();
```

```
app.UseRouting();
```

```
app.UseAuthorization();
```

```
app.MapRazorPages();
```

```
app.Run();
```

Initializes a new instance of the *WebApplicationBuilder* class. This is the starting point for configuring the application.

Key Components Initialized:

- Configuration: Sets up app configuration (e.g., reading from appsettings.json, environment variables, etc.).
- Logging: Sets up logging providers.
- Services: Registers services into the Dependency Injection (DI) container.

The Program.cs Class - 2

```
var builder = WebApplication.CreateBuilder(args);
```

```
// Add services to the container.  
builder.Services.AddRazorPages();
```

```
var app = builder.Build();
```

```
// Configure the HTTP request pipeline.  
if (!app.Environment.IsDevelopment())  
{  
    app.UseExceptionHandler("/Error");  
}  
app.UseStaticFiles();
```

```
app.UseRouting();
```

```
app.UseAuthorization();
```

```
app.MapRazorPages();
```

```
app.Run();
```

Adds Razor Pages services to the DI container.

Registers services for Razor Pages to function, including support for MVC-style patterns, routing, and Razor compilation.

Constructs the WebApplication instance based on the configured builder. This is the application that will handle incoming HTTP requests.

The Program.cs Class - 3

```
var builder = WebApplication.CreateBuilder(args);

// Add services to the container.
builder.Services.AddRazorPages();

var app = builder.Build();
```

```
// Configure the HTTP request pipeline.
if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Error");
}
app.UseStaticFiles();

app.UseRouting();

app.UseAuthorization();

app.MapRazorPages();

app.Run();
```

- The middleware pipeline handles how HTTP requests are processed and responses are generated.
 - Environment-Specific Error Handling
 - Serving Static Files
 - Enable Routing
 - Authorization Middleware
- Mapping Razor Pages
- Running the Application

Configuration In Razor Pages - 1

- ◆ ASP.NET Core includes an API for managing configuration settings needed by the application which includes a number of providers for retrieving data in a variety of different formats.
- ◆ Configuration is set up as part of the **WebHost.CreateDefaultBuilder** method called in Program.cs, the entry point to the application. Various key/value stores are added to configuration by default
 - ◆ appsettings.json
 - ◆ User Secrets (if the environment is Development)
 - ◆ Environment variables
 - ◆ Command line arguments

Configuration In Razor Pages - 2

- ◆ The *appsettings.json* file includes a section that configures logging for the application.

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft": "Warning",
      "Microsoft.Hosting.Lifetime": "Information"
    }
  },
  "AllowedHosts": "*",
  "ConnectionStrings": {
    "DefaultConnection": "Data Source=DemoDB"
  },
  "AppSettings": {
    "First": "Value 1",
    "Second": "Value 2",
    "Car": {
      "NumberOfDoors": 2,
      "RegistrationDate": "2021-08-01T00:00:00.000Z",
      "Color": "Black"
    }
  }
}
```

Configuration In Razor Pages - 3

- ◆ The *IConfiguration* object enables you to access configuration settings in a variety of ways once it has been injected into your PageModel's constructor.

- ◆ Add a using directive for ***Microsoft.Extensions.Configuration*** to the PageModel class file.

- ◆ The *Configuration* class includes a convenience method for retrieving connection strings:

- ◆ `var conn = Configuration.GetConnectionString("DefaultConnection");`

```
public class IndexModel : PageModel
{
    private readonly ILogger<IndexModel> _logger;
    private readonly IConfiguration _configuration;

    0 references
    public IndexModel(ILogger<IndexModel> logger, IConfiguration configuration)
    {
        _logger = logger;
        _configuration = configuration;
    }

    0 references
    public void OnGet()
    {
        ViewData["config"] = _configuration["AppSettings:First"];
    }
}
```

Configuration In Razor Pages - 4

- ◆ Configuring your Razor Pages site to run under HTTPS
- ◆ Running a site under HTTPS used to be something that only big online merchants worried about.
- ◆ The RequireHttps attribute is an authorization filter whose role is to confirm that requests are received over HTTPS. If the request was not made over HTTPS, the client will be redirected to the HTTPS version of the request URI if the GET method was used. Non-HTTPS requests made using any other verb (e.g. POST) will receive a 403 Forbidden result.

```
[RequireHttps]  
public class IndexModel : PageModel  
{  
    //...
```

Dependency Injection in Razor Pages

- ◆ Dependency Injection (DI) is a technique that promotes loose coupling of software through separation of concerns.
- ◆ In the context of a Razor Pages application, DI encourages you to develop discrete components for specific tasks, which are then injected into classes that need to use their functionality.
- ◆ This results in an application that is easier to maintain and test.
- ◆ Developers are advised to implement the SOLID principals of software design to ensure that their applications are robust and easier to maintain and extend. Another important guiding principal for developers is Don't Repeat Yourself (DRY), which states that should aim to reduce code repetition wherever possible.

Using Forms in Razor Pages - 1

- ◆ Forms are used for transferring data from the browser to the web server for further processing, such as saving it to a database, constructing an email, or simply subjecting the data to some kind of algorithm and then displaying the result.
- ◆ The HTML <form> element is used to create a form on a web page. The form element has a number of attributes, the most commonly used of which are method and action. The method attribute determines the HTTP verb to use when the form is submitted.
- ◆ By default, the GET verb is used and the form values are appended to the receiving page's URL as query string values. If the action attribute is omitted, the form will be submitted to the current URL i.e. the page that the form is in.

Using Forms in Razor Pages - 2

- ◆ Access the user input. User input is only available to server-side code if the form control has a value applied to the name attribute. There are several ways to reference posted form values:
 - ◆ Accessing the Request.Form collection via a string-based index, using the name attribute of the form control as the index value.

```
<form method="post">
    <input type="email" name="emailaddress">
    <input type="submit">
</form>
```

```
public void OnPost()
{
    var emailAddress = Request.Form["emailaddress"];
    // do something with emailAddress
}
```

- ◆ Leveraging Model Binding to map form fields to handler method parameters.
- ◆ Leveraging Model Binding to map form fields to public properties on a PageModel class.

Using Forms in Razor Pages - 3

- ◆ Leveraging Model Binding to map form fields to handler method parameters.

```
public void OnPost(string emailAddress)
{
    // do something with emailAddress
}
```

- ◆ Leveraging Model Binding to map form fields to public properties on a PageModel class.

```
[BindProperty]
public string EmailAddress { get; set; }
public void OnPost()
{
    // do something with EmailAddress
}
```

Validating User Input in Razor Pages – 1

- ◆ Validate user input in two places in a web application: in the browser using client-side script or the browser's in-built data type validation; and on the server.
- ◆ The MVC framework, on which Razor Pages is built, includes a robust validation framework that works against inbound model properties on the client-side and on the server.
- ◆ The key players in the input validation framework are:
 - ◆ DataAnnotation Attributes, Tag Helpers
 - ◆ jQuery Unobtrusive Validation
 - ◆ ModelState
 - ◆ Route Constraints

Validating User Input in Razor Pages – 2

- ◆ The primary building block of the validation framework is a set of attributes that inherit from *ValidationAttribute*.
- ◆ Most of these attributes reside in the *System.ComponentModel.DataAnnotations* namespace.

Attribute	Description
<code>Compare</code> ¹	Used to specify another form field that the value should be compared to for equality
<code>MaxLength</code>	Sets the maximum number of characters/bytes/items that can be accepted
<code>MinLength</code>	Sets the minimum number of characters/bytes/items that can be accepted
<code>PageRemote</code> ²	Enables client-side validation against a server-side resource, such as a database check to see if a username is already in use
<code>Range</code>	Sets the minimum and maximum values of a range
<code>RegularExpression</code>	Checks the value against the specified regular expression
<code>Remote</code> ²	Enables client-side validation against a server-side resource, such as a database check to see if a username is already in use
<code>Required</code>	Specifies that a value must be provided for this property. Note that non-nullable value types such as <code>DateTime</code> and numeric values are treated as required by default and do not need this attribute applied to them
<code>StringLength</code>	Sets the maximum, and optionally, the minimum number of string characters allowed

Validating User Input in Razor Pages – 3

- ◆ An example of DataAnnotation attributes

```
public class UserModel : PageModel
{
    [BindProperty]
    [Required]
    [MinLength(6)]
    public string UserName { get; set; }
    [BindProperty]
    [Required, MinLength(6)]
    public string Password { get; set; }
    [BindProperty, Required, Compare(nameof(Password))]
    public string Password2 { get; set; }
    ...
}
```

Validating User Input in Razor Pages – 4

- ◆ Client-side validation support is provided by the jQuery Unobtrusive Validation library, developed by Microsoft.
- ◆ Must include jQuery Unobtrusive Validation within the page containing the form for client side validation to work. This is most easily accomplished by the inclusion of the *_ValidationScriptsPartial.cshtml* file (located in the Shared folder) within the page

```
@section scripts{  
    <partial name="_ValidationScriptsPartial" />  
}
```

Validating User Input in Razor Pages – 5

- ◆ Because it is so easy to circumvent client-side validation, server-side validation is included as part of the ASP.NET Core validation framework.
- ◆ Once property values have been bound, the framework looks for all validation attributes on those properties and executes them.
- ◆ Any failures result in an entry being added to a ModelStateDictionary. This is made available in the PageModel class via *ModelState*, which has a property named *IsValid* that returns false if any of the validation tests fail.

```
public IActionResult OnPost()
{
    if (ModelState.IsValid)
    {
        // do something
        return RedirectToPage("Contact");
    }
    else
    {
        return Page();
    }
}
```

Model Binding - 1

- ◆ Model Binding in Razor Pages is the process that takes values from HTTP requests and maps them to handler method parameters or PageModel properties.
- ◆ Model binding reduces the need for the developer to manually extract values from the request and then assign them, one by one, to variables or properties for later processing.
- ◆ This work is repetitive, tedious and error prone, mainly because request values are usually only exposed via string-based indexes.
- ◆ Model binding approaches:
 - ◆ Binding posted form values to Handler Method parameters
 - ◆ Binding posted form values to PageModel properties

Model Binding - 2

- ◆ *Binding posted form values to Handler Method parameters*
- ◆ The parameters are named after the form fields, and given an appropriate type for the expected data. To see this approach in action, remove the assignment code in the OnPost handler method and add two parameters to the method

```
public void OnPost(string name, string email)
{
    ViewData["confirmation"] = $"{name}, information will be sent to {email}";
}
```

- ◆ When the form is posted, the Razor Pages framework calls the OnPost method and sees that it has two parameters.

Model Binding - 3

- ◆ *Binding posted form values to PageModel properties*
- ◆ This approach is more suitable if you need to access the values outside of the handler method (for display on the page or binding to tag helpers), or if you prefer to work in a strongly typed manner within the Razor content page.
- ◆ This approach involves adding public properties to the PageModel (or to a @functions block if you don't want to use the PageModel approach) and then decorating them with the BindProperty attribute.

Model Binding - 4

- ◆ *Binding posted form values to PageModel properties (contd.)*

```
public class ModelBindingModel : PageModel
{
    [BindProperty]
    public string Name { get; set; }
    [BindProperty]
    public string Email { get; set; }
    public void OnGet()
    {
    }
    public void OnPost()
    {
        ViewData["confirmation"] = $"{Name},
    }
}
```

```
[BindProperties]
public class ModelBindingModel : PageModel
{
    public string Name { get; set; }
    public string Email { get; set; }

    public void OnGet()
    {
    }
    public void OnPost()
    {
        ViewData["confirmation"] = $"{Name}, information will be sent to {Email}";
    }
}
```

State Management in Razor Pages - 1

- ◆ Managing state in Razor Pages is the process of retaining user or application-related data over the duration of a number of requests.
- ◆ Razor Pages, along with its underlying MVC framework provides a number of mechanisms for retaining information (or state) across requests, each with its benefits and drawbacks:
 - ◆ Hidden Form Fields
 - ◆ Query Strings
 - ◆ Route Data
 - ◆ Cookies
 - ◆ TempData, Session Variables, Application Variables, Caching

State Management in Razor Pages - 2

- ◆ Session state is a mechanism that enables you to store and retrieve user specific values temporarily. These values can be stored for the duration of the visitor's session on your site.
- ◆ Session management in ASP.NET Core is included in the *Microsoft.AspNetCore.All* metapackage, but is not enabled by default. Must enable Session State in the Startup file

```
public void ConfigureServices(IServiceCollection services)
{
    services.AddSession();
    services.AddMemoryCache();
    services.AddMvc();
}
```

```
public void Configure(IApplicationBuilder app, IHostingEnvironment env)
{
    app.UseStaticFiles();
    app.UseSession();
    app.UseMvc(routes =>
    {
        routes.MapRoute(
            name: "default",
            template: "{controller}/{action=Index}/{id?}");
    });
}
```

State Management in Razor Pages - 3

- ◆ Configuring Sessions

```
builder.Services.AddSession(options =>
{
    options.IdleTimeout = TimeSpan.FromMinutes(5);
});
```

- ◆ Using Session Variables

```
public static void SetString(this ISession session, string key, string value)
{
    session.Set(key, Encoding.UTF8.GetBytes(value));
}
```

```
public void OnGet()
{
    HttpContext.Session.SetString("Test String", "1");
    HttpContext.Session.SetInt32("Test Int", 1);
    HttpContext.Session.Set("Test Byte Array", BitConverter.GetBytes(true));
}
```

State Management in Razor Pages - 4

- ASP.NET Core uses cookies to tie multiple request together in a session.

```
builder.Services.AddSession(options =>
{
    options.Cookie.Name = "MySessionCookie";
});
```

- List of properties of Cookie

Property	Data Type	Description
Domain	string	The domain(s) that the cookie is accessible to. If it is not specified, the cookie is only sent to the domain of the current document. If provided, all sub-domains are also included.
Expiration	TimeSpan	The expiry time. While it is (currently possible)[https://github.com/aspnet/Security/issues/1293] to set an expiry time for the session cookie, you are advised not to. Session cookies should only last for the duration of the session as governed by the <code>IdleTimeout</code> setting covered at the top of this article.
HttpOnly	bool	Specifies whether the cookie is available to client-side code. false(the default) means that the cookie is accessible to JavaScript, which may present a security risk. If set to true, the cookie is only available to code executing on the server.
Name	string	The name of the cookie. The default value is <code>AspNetCore.Session</code>
Path	string	The relative path that the cookie should be accessible to. If not specified, the cookie is available to all pages in the domain(s). Subdirectories are also included. For example, /account will match /account, /account/client etc.
SameSite	SameSiteMode enum	Possible values are <code>Lax</code> (default), <code>None</code> , <code>Strict</code> . The SameSite option is intended to help in the prevention of CSRF or cookie hijacking attacks, but it is not supported by all browsers. It therefore should not be relied on.
SecurePolicy	CookieSecurePolicy enum	Specifies the security policy of the cookie. Possible values are <code>Always</code> (Secure is always marked <code>true</code>), <code>None</code> (Secure is not marked <code>true</code>) and <code>SameAsRequest</code> (The default: the cookie will be sent using the protocol of the request).

Working with AJAX in Razor Pages - 1

- ◆ AJAX is a technique used for making requests from the browser to the server for various purposes such as the retrieval of assorted content including data, HTML, XML, posting form values and so on.
- ◆ Requests are initiated from client script (usually JavaScript) once the page has already been loaded in the browser.
- ◆ If you wanted to update parts of the page as a result of choices that the user made, those choices would have to be sent to the server as a form post and the page would be regenerated on the web server in its entirety.
- ◆ The use of AJAX in a web page eliminates this stop-start approach to working in a web page.

```
() => {  
    // code to execute  
}
```

Working with AJAX in Razor Pages - 2

```
public class Car
```

```
{  
1 public int Id { get; set; }  
  public string Make { get; set; }  
  public string Model { get; set; }  
  public int Year { get; set; }  
  public int Doors { get; set; }  
  public string Colour { get; set; }  
  public decimal Price { get; set; }  
}
```

2

```
public interface ICarService  
{  
  List<Car> GetAll();  
}
```

```
public class CarService : ICarService  
{
```

```
  public List<Car> GetAll()  
  {
```

```
    List<Car> cars = new List<Car>{  
      new Car{Id = 1, Make="Audi",Model="R8",Year=2014,Doors=2,Colour="Red",Price=79995},  
      new Car{Id = 2, Make="Aston Martin",Model="Rapid",Year=2010,Doors=2,Colour="Black",Price=54995},  
      new Car{Id = 3, Make="Porsche",Model="911 991",Year=2016,Doors=2,Colour="White",Price=155000},  
      new Car{Id = 4, Make="Mercedes-Benz",Model="GLE 63S",Year=2017,Doors=5,Colour="Blue",Price=83995},  
    },  
    new Car{Id = 5, Make="BMW",Model="X6 M",Year=2016,Doors=5,Colour="Silver",Price=62995},  
  };  
  return cars;  
}
```

Working with AJAX in Razor Pages - 3

3

```
builder.Services.AddTransient<ICarService, CarService>();
```

5

```
public class AjaxPartialModel : PageModel
{
    private ICarService _carService;
    public AjaxPartialModel(ICarService carService)
    {
        _carService = carService;
    }
    public List<Car> Cars { get; set; }
    public void OnGet()
    {
    }
    public PartialViewResult OnGetCarPartial()
    {
        Cars = _carService.GetAll();
        return Partial("_CarPartial", Cars);
    }
}
```

4

```
@model List<Car>
<table class="table table-striped">
@foreach(var car in Model)
{
    <tr>
        <td>@car.Model</td>
        <td>@car.Make</td>
        <td>@car.Year</td>
        <td>@$@car.Price</td>
    </tr>
}
</table>
```

6

```
public PartialViewResult OnGetCarPartial()
{
    Cars = _carService.GetAll();
    return new PartialViewResult {
        ViewName = "_CarPartial",
        ViewData = new ViewDataDictionary<List<Car>>(ViewData, Cars)
    };
}
```

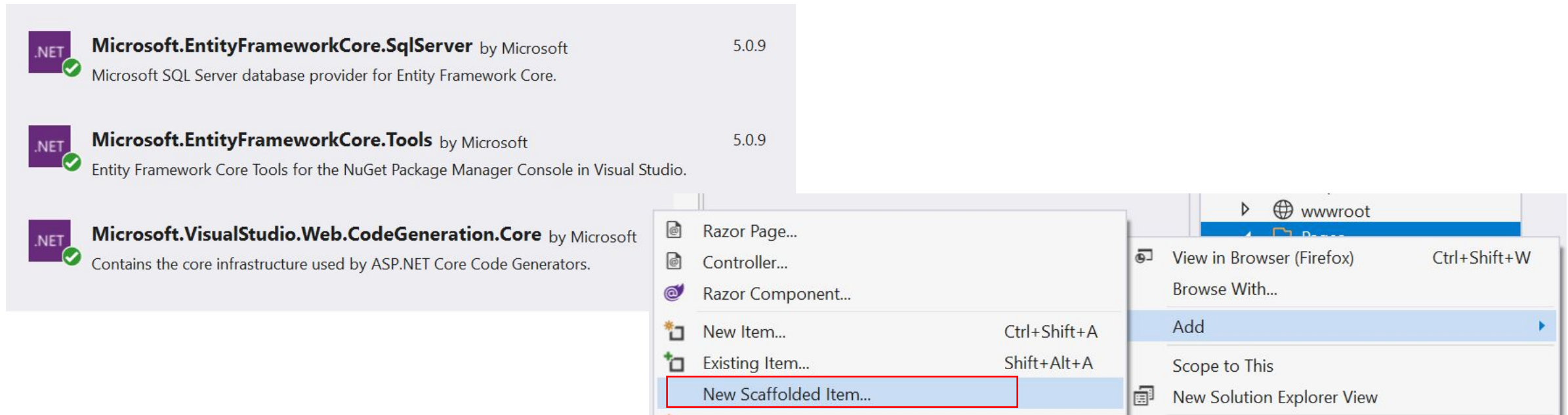
Working with AJAX in Razor Pages - 3

7

```
@page
@model RazorPagesTests.Pages.AjaxPartialModel
<h2>Ajax</h2>
<p><button class="btn btn-primary" id="load">Load</button></p>
<div id="grid"></div>
@section scripts{
    <script>
        document.getElementById('load').addEventListener('click', () => {
            fetch('/ajaxpartial?handler=CarPartial')
                .then((response) => {
                    return response.text();
                })
                .then((result) => {
                    document.getElementById('grid').innerHTML = result;
                });
        });
    </script>
}
```

Scaffolding Razor Pages

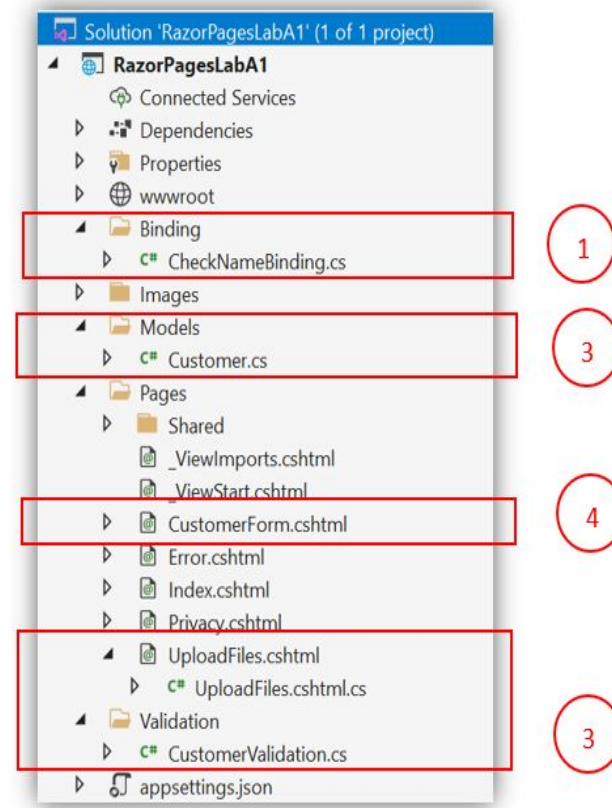
- ◆ Scaffolding in ASP.NET Core is a technique used to generate code at design time to support a number of common application scenarios when working with Entity Framework Core.
- ◆ The code generation tool is available as a Nuget package.



Razor Pages Demo

Demo 1.

- ◆ **Step 01.** Create ASP.NET Core Web App
- ◆ **Step 02.** Using Model Binding in Razor Pages to takes values from HTTP requests and maps them to handler method parameters or PageModel properties.
- ◆ **Step 03.** Create custom validation class
- ◆ **Step 04.** Create a Model using DataAnnotations and custom validation.
- ◆ **Step 05.** Create Razor Page (Empty) for form validation
- ◆ **Step 06.** Create Razor Page (Empty) for uploading files
- ◆ **Step 07.** Build and run Project.



Demo 1.

◆ **Step 01.** Create ASP.NET Core Web App (A project template for creating an ASP.NET application with example ASP.NET Razor Pages content.)

The image displays three sequential screenshots of the Visual Studio project creation interface:

- Top Left Screenshot:** The "Create a new project" window. Under "Recent project templates", the "ASP.NET Core Web App" template is highlighted with a red box. The template description states: "A project template for creating an ASP.NET Core application with example ASP.NET Razor Pages content." The "C#" language and "Web" category are selected.
- Bottom Left Screenshot:** The "Configure your new project" window. The "Project name" field is set to "RazorPagesLabA1". The "Location" is "C:\Users\HP\source\repos". The "Solution" is "Create new solution". The "Solution name" is "RazorPagesLabA1". The "Next" button at the bottom right is highlighted with a red box.
- Right Screenshot:** The "Additional information" window. The "Target Framework" is set to ".NET 5.0 (Current)". The "Authentication Type" is "None". The "Configure for HTTPS" checkbox is checked. The "Create" button at the bottom right is highlighted with a red box.

Demo 1.

- ◆ **Step 02.** Using Model Binding in Razor Pages to takes values from HTTP requests and maps them to handler method parameters or PageModel

```

1 using Microsoft.AspNetCore.Mvc.ModelBinding;
2 using Microsoft.Extensions.Logging;
3 using System;
4 using System.Threading.Tasks;
5
6 namespace RazorPagesLabA1.Binding
7 {
8     4 references
9     public class CheckNameBinding : IModelBinder
10     {
11         private readonly ILogger<CheckNameBinding> _logger;
12
13         0 references
14         public CheckNameBinding(ILogger<CheckNameBinding> logger)
15         {
16             _logger = logger;
17         }
18
19         0 references
20         public Task BindModelAsync(ModelBindingContext bindingContext)
21         {
22             if (bindingContext == null)
23             {
24                 throw new ArgumentNullException(nameof(bindingContext));
25             }
26             // Get ModelName
27             string modelName = bindingContext.ModelName;
28             ValueProviderResult valueProviderResult = bindingContext.ValueProvider.GetValue(modelName);
29         }
30     }
31 }

```

```

26
27
28
29
30
31 // Set ModelState for the value binding
32 bindingContext.ModelState.SetModelValue(modelName, valueProviderResult);
33
34 string value = valueProviderResult.FirstValue;
35 if (string.IsNullOrEmpty(value))
36 {
37     return Task.CompletedTask;
38 }
39 var s = value.ToUpper();
40
41 if (s.Contains("XXX"))
42 {
43     bindingContext.ModelState.TryAddModelError(
44         modelName, "Cannot contain this pattern xxx.");
45     return Task.CompletedTask;
46 }
47 bindingContext.Result = ModelBindingResult.Success(s.Trim());
48
49 return Task.CompletedTask;
50
51 }
52
53
54

```

Demo 1.

- ◆ **Step 03.** Create custom validation class

```
1  using System;
2  using System.ComponentModel.DataAnnotations;
3
4  namespace RazorPagesLabA1.Validation
5  {
6      2 references
7      public class CustomerValidation:ValidationAttribute
8      {
9          1 reference
10         public CustomerValidation()
11         {
12             ErrorMessage = "The year of birth cannot greeter than current year (2021).";
13         }
14         0 references
15         public override bool IsValid(object value)
16         {
17             if (value == null)
18                 return false;
19             int number = Int32.Parse(value.ToString());
20             return (number < 2021);
21         }
22     }
```

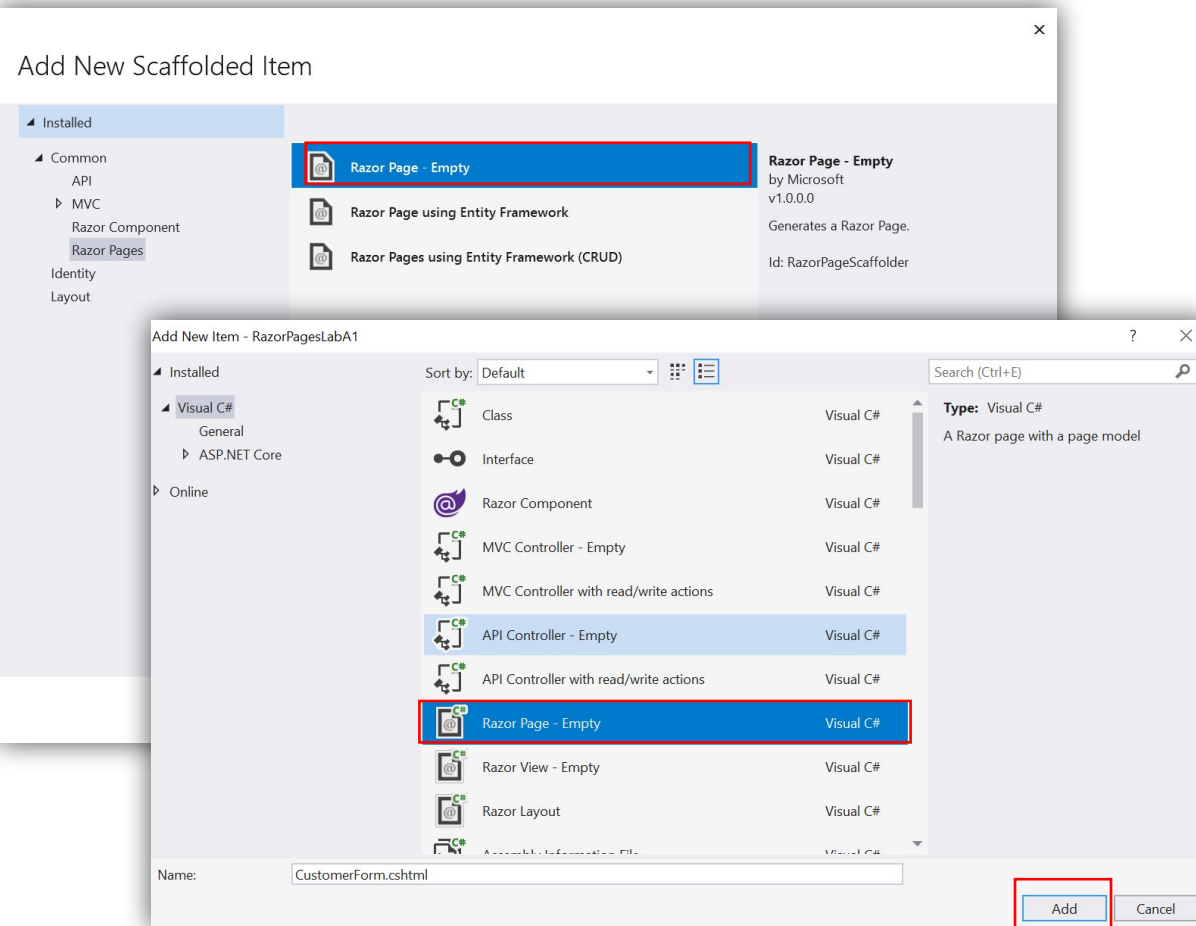
Demo 1.

- ◆ **Step 04.** Create a Model using DataAnnotations and custom validation.

```
1 using Microsoft.AspNetCore.Mvc;
2 using RazorPagesLabA1.Validation;
3 using RazorPagesLabA1.Binding;
4 using System;
5 using System.ComponentModel.DataAnnotations;
6
7 namespace RazorPagesLabA1.Models
8 {
9     1 reference
10     public class Customer
11     {
12         [Required(ErrorMessage = "Customer name is required!")]
13         [StringLength(20, MinimumLength = 3, ErrorMessage = "The length of name is from 3 to 20 chara")]
14         [Display(Name = "Customer name")]
15         [ModelBinder(BinderType = typeof(CheckNameBinding))]
16         4 references
17         public string CustomerName { set; get; }
18
19         [Required(ErrorMessage = "Customer email is required!")]
20         [EmailAddress]
21         [Display(Name = "Customer email")]
22         4 references
23         public string Email { set; get; }
24
25         [Required(ErrorMessage = "Year of birth is required!")]
26         [Display(Name = "Year of birth")]
27         [Range(1960, 2000, ErrorMessage = "1960 - 2000")]
28         [CustomerValidation]
29         4 references
30         public int? YearOfBirth { set; get; }
31     }
32 }
```

Demo 1.

Step 05. Create Razor Page (Empty)



```
1 @page
2 @model RazorPagesLabA1.Pages.CustomerFormModel
3
4 var customerinfo = Model.customerInfo;
5
6 if (customerinfo != null)
7 {
8     <h3>@Model.Message</h3>
9     <div asp-validation-summary="All"></div> if (ModelState.IsValid)
10     {
11         <div class="card">
12             <div class="card-body">
13                 <p><strong>Customer name: </strong> @customerinfo.CustomerName</p>
14                 <p><strong>Customer email: </strong> @customerinfo.Email</p>
15                 <p><strong>Year Of Birth: </strong> @customerinfo.YearOfBirth</p>
16             </div>
17         </div>
18     }
19 }
20
21
22 <h3>Customer information</h3>
23 <form method="post" class="m-2 p-3 bg-light">
24     <div class="form-group">
25         <label asp-for="@customerinfo.CustomerName"></label>
26         <input class="form-control" asp-for="@customerinfo.CustomerName" />
27         <span asp-validation-for="@customerinfo.CustomerName" class="text-danger"></span>
28     </div>
29     <div class="form-group">
30         <label asp-for="@customerinfo.Email"></label>
31         <input class="form-control" asp-for="@customerinfo.Email" />
32         <span asp-validation-for="@customerinfo.Email" class="text-danger"></span>
33     </div>
34     <div class="form-group">
35         <label asp-for="@customerinfo.YearOfBirth"></label>
36         <input class="form-control" asp-for="@customerinfo.YearOfBirth" />
37         <span asp-validation-for="@customerinfo.YearOfBirth" class="text-danger"></span>
38     </div>
39     <div class="form-group">
40         <button class="btn btn-primary" asp-page="Form">Send</button>
41     </div>
42 </form>
```

Demo 1.

Step 05. (contd.)

```
1 using Microsoft.AspNetCore.Mvc;
2 using Microsoft.AspNetCore.Mvc.RazorPages;
3 using RazorPagesLabA1.Models;
4
5 namespace RazorPagesLabA1.Pages
6 {
7     5 references
8     public class CustomerFormModel : PageModel
9     {
10         3 references
11         public string Message { set; get; }
12         [BindProperty]
13         1 reference
14         public Customer customerInfo { set; get; }
15
16         0 references
17         public void OnPost()
18         {
19             if (ModelState.IsValid)
20             {
21                 Message = "Information is OK";
22                 ModelState.Clear();
23             }
24             else
25             {
26                 Message = "Error on input data.";
27             }
28         }
29     }
30 }
```

Demo 1.

◆ Step 06. Create Razor Page (Empty) for uploading files

```

1  @page
2  @model RazorPagesLabA1.Pages.UploadFilesModel
3  @{
4      <form method="post" enctype="multipart/form-data">
5          <label asp-for="@Model.FileUploads"></label>
6          <input asp-for="@Model.FileUploads" />
7          <span asp-validation-formaction="@Model.FileUploads"></span>
8
9          <button class="btn btn-primary" asp-page="UploadMulti">Upload</button>
10     </form>
11 }

```

```

1  using System.ComponentModel.DataAnnotations;
2  using System.IO;
3  using System.Threading.Tasks;
4  using Microsoft.AspNetCore.Hosting;
5  using Microsoft.AspNetCore.Http;
6  using Microsoft.AspNetCore.Mvc;
7  using Microsoft.AspNetCore.Mvc.RazorPages;
8
9
10 namespace RazorPagesLabA1.Pages
11 {
12     6 references
13     public class UploadFilesModel : PageModel
14     {
15         private IHostingEnvironment _environment;
16         0 references
17         public UploadFilesModel(IHostingEnvironment environment)
18         {
19             _environment = environment;
20
21         [Required(ErrorMessage = "Please choose at least one file.")]
22         [DataType(DataType.Upload)]
23         [FileExtensions(Extensions = "png,jpg,jpeg,gif")]
24         [Display(Name = "Choose file(s) to upload")]
25         [BindProperty]
26         5 references
27         public IFormFile[] FileUploads { get; set; }
28
29     0 references
30     public async Task OnPostAsync()
31     {
32         if (FileUploads != null)
33         {
34             foreach (var FileUpload in FileUploads)
35             {
36                 var file = Path.Combine(_environment.ContentRootPath, "Images",
37                                         FileUpload.FileName);
38                 using (var fileStream = new FileStream(file, FileMode.Create))
39                 {
40                     await FileUpload.CopyToAsync(fileStream);
41                 }
42             }
43         }
44     }
45 }

```

Demo 1.

Customer information

Customer name

Customer email

Year of birth

Send

Shipping Tool

Error on input data.

- Customer name is required!
- Customer email is required!
- Year of birth is required!
- The year of birth cannot greeter than current year (2021).

Customer information

Customer name

Customer name is required!

Customer email

Customer email is required!

Year of birth

Year of birth is required!

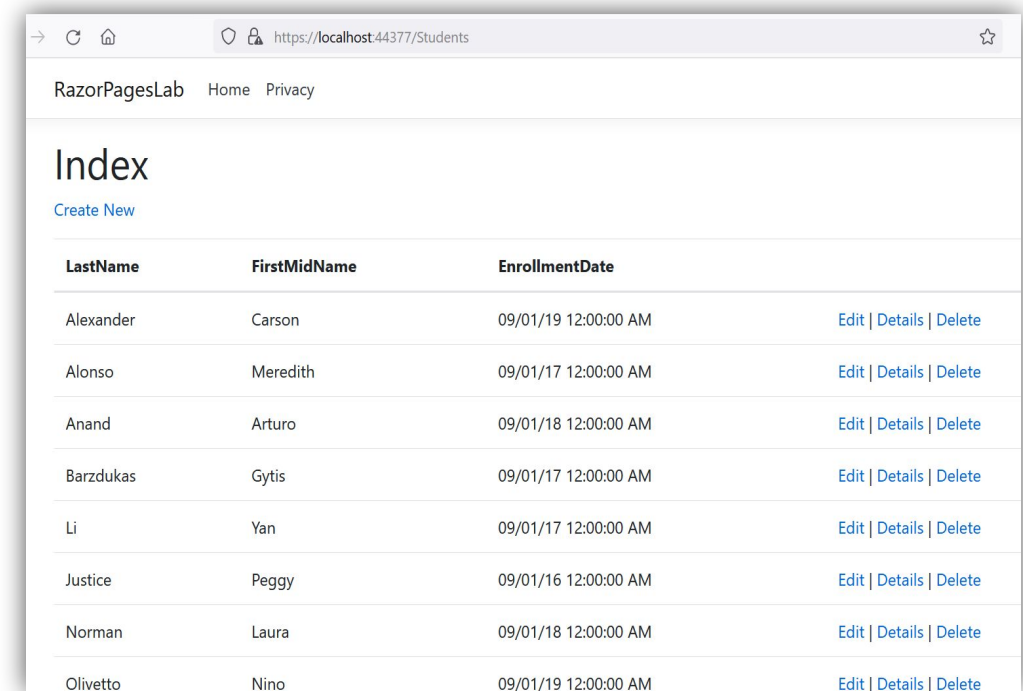
Send

Choose file(s) to upload **Browse...** No files selected.

Upload

Demo 2. Razor Pages with Entity Framework

- ◆ **Step 01.** Create ASP.NET Core Web Application
- ◆ **Step 02.** Add model – Student, Course, Enrolment.
- ◆ **Step 03.** Manage NuGet packages for Solution/Project
- ◆ **Step 04.** Add Connection string (appsettings.json file)
- ◆ **Step 05.** Scaffold Student pages
- ◆ **Step 06.** Change the code on Startup.cs and Program.cs
- ◆ **Step 07.** Build and run Program.



RazorPagesLab Home Privacy

Index

[Create New](#)

LastName	FirstMidName	EnrollmentDate	
Alexander	Carson	09/01/19 12:00:00 AM	Edit Details Delete
Alonso	Meredith	09/01/17 12:00:00 AM	Edit Details Delete
Anand	Arturo	09/01/18 12:00:00 AM	Edit Details Delete
Barzdukas	Gytis	09/01/17 12:00:00 AM	Edit Details Delete
Li	Yan	09/01/17 12:00:00 AM	Edit Details Delete
Justice	Peggy	09/01/16 12:00:00 AM	Edit Details Delete
Norman	Laura	09/01/18 12:00:00 AM	Edit Details Delete
Olivetto	Nino	09/01/19 12:00:00 AM	Edit Details Delete

Demo 2. Razor Pages with Entity Framework

- ◆ Manage NuGet packages for Solution/Project

The screenshot displays the Visual Studio NuGet Package Manager interface. On the left, under 'Top-level packages (2)', two packages are listed:

- Microsoft.EntityFrameworkCore.SqlServer** 8.0.11: Microsoft SQL Server database provider for Entity Framework Core. 9.0.0
- Microsoft.EntityFrameworkCore.Tools** 8.0.11: Entity Framework Core Tools for the NuGet Package Manager Console in Visual Studio. 9.0.0

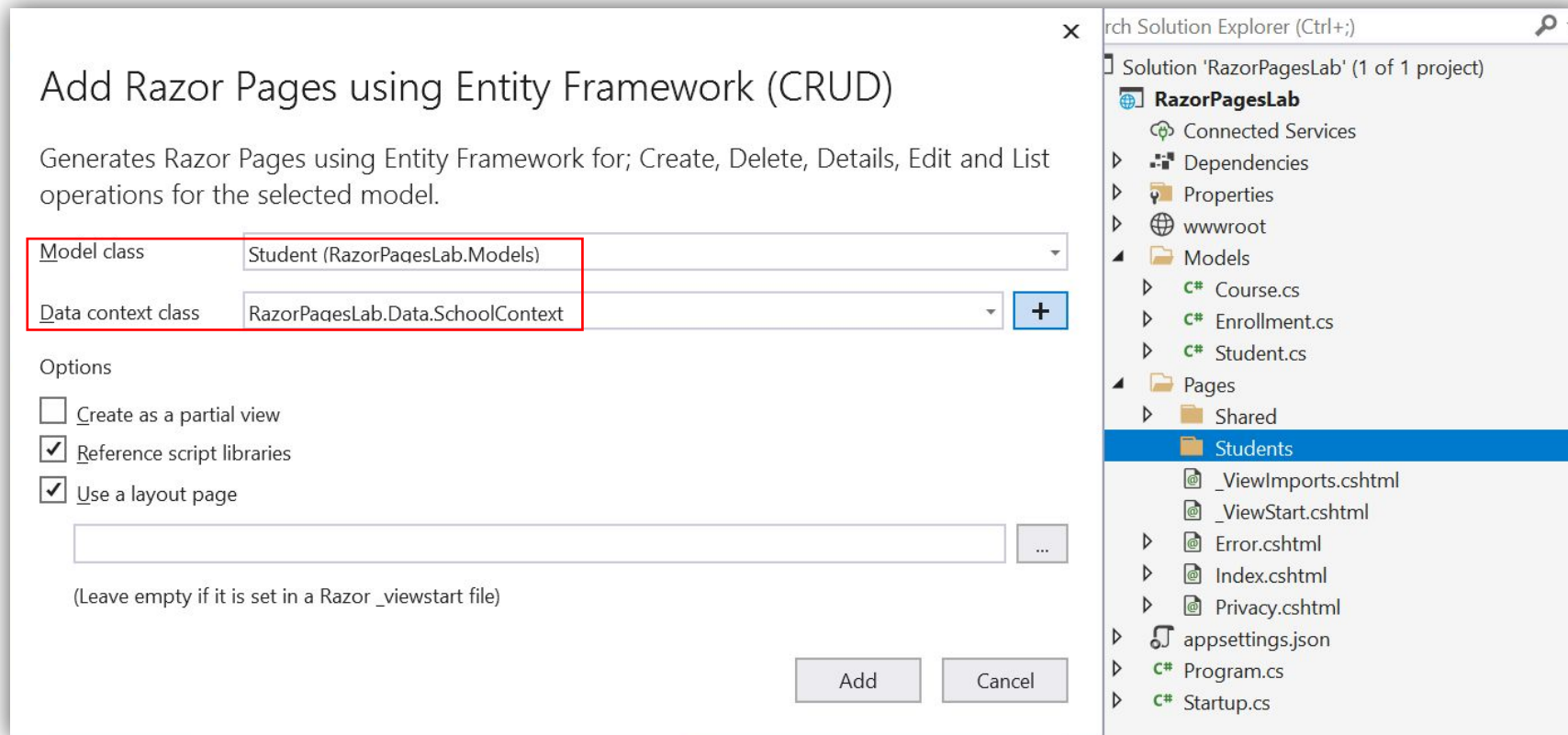
The center pane shows the details for **Microsoft.EntityFrameworkCore** from **nuget.org**. It indicates the package is **Installed: 8.0.11** and the **Version: 8.0.11**. Buttons for 'Uninstall' and 'Update' are visible. A note states 'Package source mapping is off. [Configure](#)'. An 'Options' button is at the bottom.

The right pane shows the 'Dependencies' tree. Under 'Packages', two items are listed and highlighted with a red box:

- Microsoft.EntityFrameworkCore.SqlServer (8.0.11)
- Microsoft.EntityFrameworkCore.Tools (8.0.11)

Demo 2. Razor Pages with Entity Framework

The scaffolding process will provide these files (creates Razor pages in the *Pages/Students* folder)



Demo 3

- Razor Pages with Entity Framework CRUD (includes search, sorting and paging)

The screenshot shows a web browser window displaying a page titled "Students" from "RazorPagesLab". The page includes a search bar with the text "Find by name:", a "Search" button, and a link "Back to full List". Below the search bar is a table with three columns: "LastName", "FirstMidName", and "EnrollmentDate". The table contains three rows of student data. At the bottom of the table are "Previous" and "Next" buttons for pagination. The footer of the page reads "© 2021 - RazorPagesLab - Privacy".

LastName	FirstMidName	EnrollmentDate	
Alexander	Carson	09/01/19 12:00:00 AM	Edit Details Delete
Alonso	Meredith	09/01/17 12:00:00 AM	Edit Details Delete
Anand	Arturo	09/01/18 12:00:00 AM	Edit Details Delete

Summary

- ◆ Concepts were introduced:
 - ◆ Razor Page files and Razor syntax
 - ◆ Page Models (Handler Methods, ViewData, Action Results)
 - ◆ Tag Helpers
 - ◆ View Components
 - ◆ Routing and URLs
 - ◆ Startup with Razor Pages and Configuration
 - ◆ Validation and Model Binding
 - ◆ State Management
 - ◆ Scaffolding